ROBO AI

INDUSTRIAL TRAINING PROGRAM ON

ROBOTICS AND AI

PROJECT REPORT

IoT-BASED WEATHER STATION USING ESP32 AND CLOUD DASHBOARD

Submitted as part of the Industrial
Training Program on Robotics and AI

PROJECT DETAILS

Project Domain: Internet of Things (IoT)
Microcontroller: ESP32
Sensor: DHT22 – Temperature and Humidity
Cloud Platform: ThingSpeak
Programming Language: Embedded C++
Simulation Platform: Wokwi
Alert Mechanism: ThingSpeak Alerts API and MATLAB Analysis

TECHNOLOGIES USED

ESP32 | DHT22 | Wi-Fi | Arduino C++ | ThingSpeak | MATLAB | Wokwi

---

SUBMITTED BY

SHUBHAM DATTATRAY ASOLE

Google Drive – Open Project Files and Evidence

Abstract

The **IoT-Based Weather Station Using ESP32 and Cloud Dashboard** is designed to monitor environmental conditions, particularly temperature and relative humidity, using Internet of Things (IoT) technology. The system acquires sensor readings, transmits them to a cloud platform, visualizes the measurements and generates email notifications when configured threshold values are exceeded.

The design uses an ESP32 microcontroller interfaced with a DHT22 temperature and humidity sensor. The ESP32 reads the sensor data using Arduino C++ firmware and connects to the internet through Wi-Fi. The readings are transmitted to the ThingSpeak cloud platform using HTTP communication. ThingSpeak stores the measurements in separate channel fields and presents them as time-based graphs.

An email notification mechanism is implemented using ThingSpeak Alerts API and MATLAB Analysis. The configured alert conditions are temperature above 35 °C and relative humidity above 70%. Both temperature and humidity alert emails were received during testing.

The complete hardware path was implemented as a simulation in Wokwi. The project demonstrates sensor interfacing, Wi-Fi connectivity, cloud data upload, graphical visualization and threshold-based email notification. Since no physical components were used, the results represent a simulation-based proof of concept rather than physical hardware validation.

Keywords: IoT, ESP32, DHT22, ThingSpeak, Wokwi, Wi-Fi, MATLAB Analysis, Alerts API, temperature monitoring, humidity monitoring.

Contents

Abstract **i**

1 Introduction **1**
1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 1
1.4 Scope . . . 2
1.5 Expected Outcomes . . . 2

2 System Architecture and Design **3**
2.1 Overall Architecture . . . 3
2.2 System Operation . . . 3
2.3 System Configuration . . . 4
2.4 Data Flow . . . 4

3 Components and Software Tools **5**
3.1 ESP32 Development Board . . . 5
3.2 DHT22 Sensor . . . 5
3.3 ThingSpeak Cloud Platform . . . 5
3.4 Wokwi Simulation Platform . . . 5
3.5 Software and Libraries . . . 6

4 Circuit Design and Sensor Interfacing **7**
4.1 Circuit Connections . . . 7
4.2 Circuit Diagram . . . 7
4.3 Sensor Reading Logic . . . 8

5 ESP32 Firmware Implementation **9**
5.1 Firmware Overview . . . 9
5.2 Program Logic . . . 9
5.3 ESP32 Source Code . . . 9
5.4 Library Dependencies . . . 13

6 ThingSpeak Cloud Integration and Visualization **14**
6.1 Channel Configuration . . . 14
6.2 Data Upload . . . 14
6.3 Data Visualization . . . 14
6.4 Upload Interval . . . 16

7 Email Alert Mechanism **17**
7.1 Alert Conditions . . . 17
7.2 MATLAB Analysis . . . 17
7.3 Temperature Alert Code . . . 17
7.4 Humidity Alert Code . . . 18
7.5 Alert API Rate Limit . . . 18
7.6 Email Verification . . . 19

8 Testing, Results and Discussion **21**
8.1 Testing Methodology . . . 21
8.2 Functional Test Cases . . . 21
8.3 Results . . . 22
8.4 Discussion . . . 22
8.5 Screenshots and Evidence . . . 22

9 Limitations and Future Scope **24**
9.1 Limitations . . . 24
9.2 Future Enhancements . . . 24

10 Conclusion **25**

References **26**

List of Figures

2.1 Project block diagram. 3

4.1 Wokwi circuit diagram showing the ESP32 and DHT22 wiring. 8

6.1 ThingSpeak temperature readings (Field 1). 15
6.2 ThingSpeak relative humidity readings (Field 2). 15

7.1 Received high-temperature email notification. 19
7.2 Received high-humidity email notification. 20

8.1 ESP32 Serial Monitor showing temperature, humidity, Wi-Fi connection and successful ThingSpeak data upload. 23

List of Tables

2.1 Project configuration 4

3.1 Software tools used 6

4.1 DHT22 to ESP32 connections 7

7.1 Configured alert thresholds 17

8.1 Functional test cases 21

CHAPTER 1

Introduction

1.1 Background

Environmental parameters such as temperature and relative humidity are useful for understanding local atmospheric conditions. A connected monitoring system can collect these measurements periodically and make them available remotely. Internet of Things (IoT) systems combine sensors, embedded controllers, communication networks and cloud services to support this type of monitoring.

This project demonstrates a small IoT weather station using an ESP32 and a DHT22 sensor. The sensor provides temperature and relative humidity measurements. The ESP32 reads the sensor and sends the measurements to ThingSpeak over Wi-Fi. The cloud service stores the values and displays them as graphs. A separate alert workflow uses MATLAB Analysis and the ThingSpeak Alerts API to send email notifications when configured thresholds are exceeded.

1.2 Problem Statement

Traditional observation may require a person to read a local instrument and manually record the measured values. Such a process does not inherently provide remote access to the data or automatic notification when a measured value exceeds a chosen limit.

The problem addressed in this project is to demonstrate an IoT-based system that can acquire temperature and humidity readings, transmit them wirelessly to a cloud service, display the readings over time and issue email notifications when predefined limits are crossed. The implementation is simulated because physical ESP32 and DHT22 components were not available.

1.3 Objectives

The objectives of the project are:

1. Interface a DHT22 temperature and humidity sensor with an ESP32.

2. Acquire temperature in degrees Celsius and relative humidity in percent.
3. Connect the ESP32 to a Wi-Fi network.
4. Upload sensor measurements to a ThingSpeak channel.
5. Visualize temperature and humidity as separate cloud graphs.
6. Configure email notifications for temperature above 35 °C and humidity above 70%.
7. Verify the complete data flow through Wokwi simulation and cloud services.

1.4 Scope

The scope includes the simulated sensor circuit, ESP32 firmware, Wi-Fi communication, ThingSpeak channel configuration, data visualization and email alert demonstration. The project does not include physical enclosure design, calibration against a reference instrument, outdoor deployment or long-term field testing.

1.5 Expected Outcomes

The expected outcomes are successful virtual sensor readings, successful upload of data to ThingSpeak, visible temperature and humidity graphs, and receipt of alert emails when the configured conditions are met.

CHAPTER 2

System Architecture and Design

2.1 Overall Architecture

The system contains a sensing unit, processing and communication unit, cloud data service, visualization interface and alert service. The DHT22 provides digital measurements to the ESP32. The ESP32 uses Wi-Fi to send the values to ThingSpeak. ThingSpeak stores the values and displays them. MATLAB Analysis and the Alerts API are used for the email notification workflow.

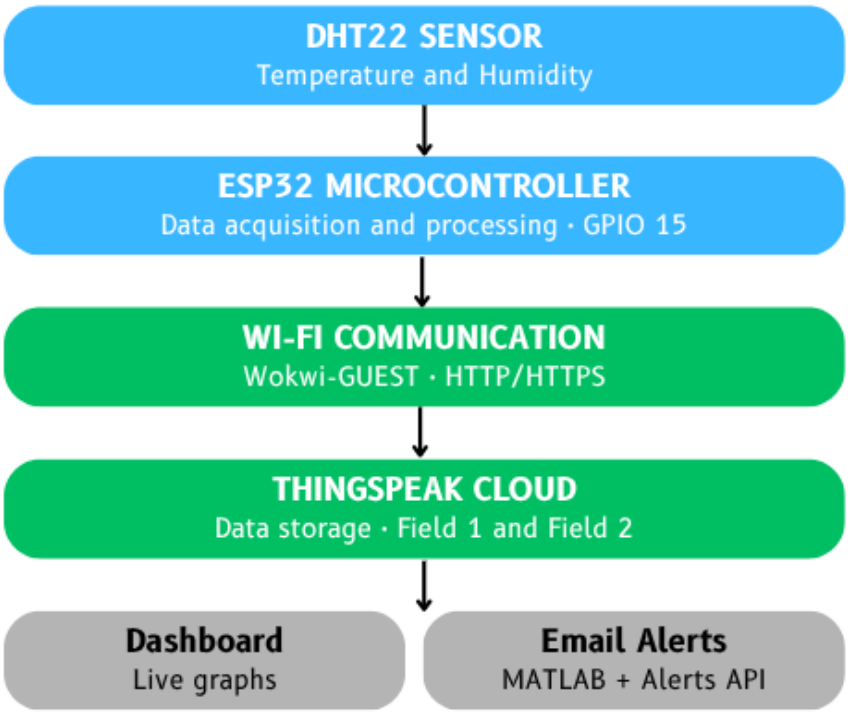


Figure 2.1: Project block diagram.

2.2 System Operation

The system operates in the following sequence:

1. The virtual DHT22 sensor provides temperature and humidity values.

2. The ESP32 reads the sensor through GPIO 15.
3. The firmware checks that the sensor readings are valid.
4. The ESP32 connects to the Wokwi virtual Wi-Fi network.
5. The firmware sends temperature and humidity values to ThingSpeak.
6. ThingSpeak stores the values in Field 1 and Field 2.
7. The channel charts display the uploaded measurements.
8. MATLAB Analysis sends an alert request when the configured threshold condition is met.

2.3 System Configuration

Table 2.1: Project configuration

Parameter	Configuration
Microcontroller	ESP32 DevKit (Wokwi simulation)
Sensor	DHT22
Sensor data connection	GPIO 15
Sensor supply connection	3.3 V
Ground	ESP32 GND
Wi-Fi SSID	Wokwi-GUEST
Cloud platform	ThingSpeak
Channel fields	Field 1: Temperature; Field 2: Humidity
Upload interval	20 seconds in the provided firmware
Temperature threshold	Above 35 °C
Humidity threshold	Above 70%
Simulation platform	Wokwi

2.4 Data Flow

The DHT22 measurement is read by the ESP32 and represented as two numerical values: temperature in degrees Celsius and relative humidity in percent. These values are transmitted to the ThingSpeak channel. The cloud channel stores each upload as a new entry, allowing the values to be inspected as a function of time.

CHAPTER 3

Components and Software Tools

3.1 ESP32 Development Board

The ESP32 is a microcontroller platform with integrated Wi-Fi capability. In this project it reads the DHT22 sensor and provides network connectivity for uploading measurements to ThingSpeak. The implementation uses GPIO 15 as the DHT22 data connection.

3.2 DHT22 Sensor

The DHT22 is a digital temperature and relative humidity sensor. It provides the two environmental measurements used by this project. The sensor is powered from the ESP32 3.3 V pin in the simulated circuit. Its data pin is connected to GPIO 15 and its ground pin is connected to GND.

3.3 ThingSpeak Cloud Platform

ThingSpeak is used as the cloud destination for the sensor readings. A channel is configured with two fields: Field 1 for temperature and Field 2 for humidity. The channel's private view provides the graphs used to observe uploaded data.

3.4 Wokwi Simulation Platform

Wokwi is an online electronics simulator used to create the ESP32 and DHT22 circuit and run the firmware without physical components. The virtual sensor controls allow the temperature and humidity values to be changed during testing.

3.5 Software and Libraries

Table 3.1: Software tools used

Tool / Library	Purpose
Wokwi	Simulates ESP32 and DHT22 hardware
Arduino C++	Firmware development
DHT sensor library	Reads DHT22 measurements
WiFi.h	Connects ESP32 to Wi-Fi
HTTPClient.h	Performs HTTP requests
WiFiClientSecure.h	Provides secure client transport for HTTPS
ThingSpeak	Stores and visualizes sensor data
MATLAB Analysis	Evaluates data and initiates alert requests
ThingSpeak Alerts API	Sends email alert requests

CHAPTER 4

Circuit Design and Sensor Interfacing

4.1 Circuit Connections

The DHT22 sensor is connected to the ESP32 as shown in Table 4.1. The NC pin is left unconnected.

Table 4.1: DHT22 to ESP32 connections

DHT22 pin	ESP32 connection	Description
VCC	3V3	Sensor supply
SDA / DATA	GPIO 15	Digital data signal
NC	Not connected	No connection required
GND	GND	Common ground

4.2 Circuit Diagram

The following diagram shows the logical wiring used in the Wokwi simulation. It is a connection schematic rather than a photograph of physical hardware.

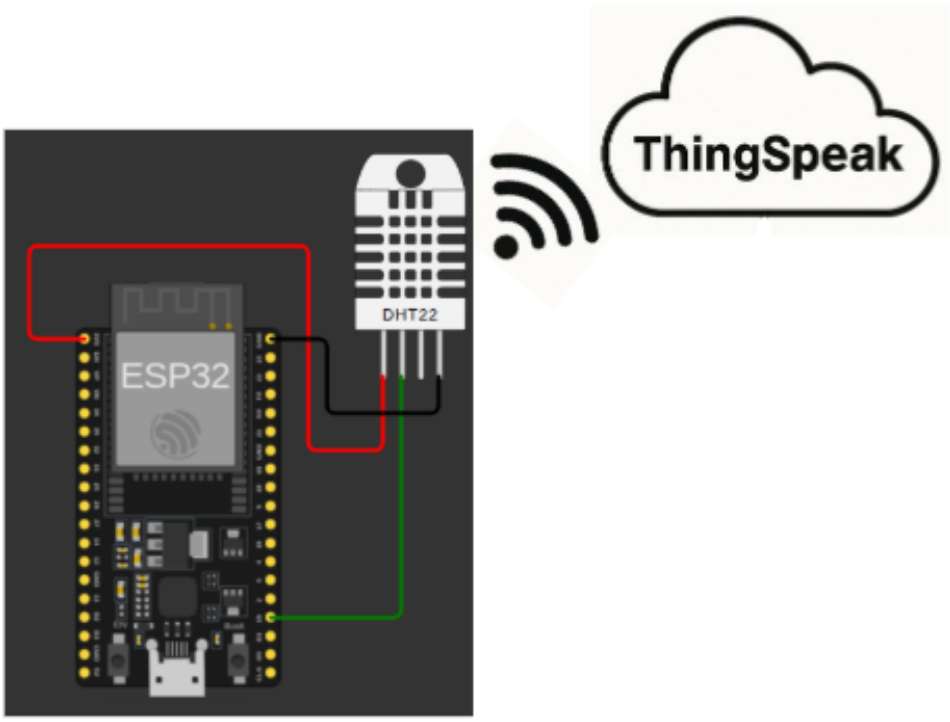


Figure 4.1: Wokwi circuit diagram showing the ESP32 and DHT22 wiring.

4.3 Sensor Reading Logic

The DHT library is initialized in the setup routine. During each measurement cycle, the firmware calls the temperature and humidity reading functions. The returned values are checked using `isnan()` to detect invalid readings. Valid readings are printed to the Serial Monitor and passed to the cloud upload routine.

CHAPTER 5

ESP32 Firmware Implementation

5.1 Firmware Overview

The firmware performs four principal operations: initialization of serial communication and the DHT22 sensor, connection to Wi-Fi, acquisition of temperature and humidity, and periodic upload to ThingSpeak. The configured upload interval is 20 seconds.

5.2 Program Logic

1. Start serial communication at 115200 baud.
2. Initialize the DHT22 sensor on GPIO 15.
3. Connect to the Wokwi-GUEST Wi-Fi network.
4. At each upload interval, read temperature and humidity.
5. Check whether either reading is invalid.
6. If the readings are valid, transmit them to ThingSpeak.
7. Print the HTTP response and returned ThingSpeak entry identifier.

5.3 ESP32 Source Code

Note: Replace the placeholder with the private Write API Key from the ThingSpeak channel before running the program. Never publish a real API key in a public source repository or report screenshot.

Listing 5.1: ESP32 firmware for DHT22 and ThingSpeak upload

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include "DHT.h"
```

```
5
6  #define DHTPIN 15
7  #define DHTTYPE DHT22
8
9  DHT dht(DHTPIN, DHTTYPE);
10
11 const char* WIFI_SSID = "Wokwi-GUEST";
12 const char* WIFI_PASSWORD = "";
13
14 const char* WRITE_API_KEY =
15     "PASTE_YOUR_WRITE_API_KEY_HERE";
16
17 const char* SERVER_URL =
18     "https://api.thingspeak.com/update";
19
20 const unsigned long UPLOAD_INTERVAL = 20000;
21 unsigned long lastUploadTime = 0;
22
23 void connectWiFi() {
24   Serial.print("Connecting to WiFi");
25   WiFi.begin(WIFI_SSID, WIFI_PASSWORD, 6);
26
27   while (WiFi.status() != WL_CONNECTED) {
28     delay(500);
29     Serial.print(".");
30   }
31
32   Serial.println();
33   Serial.println("WiFi Connected!");
34   Serial.print("IP Address: ");
35   Serial.println(WiFi.localIP());
36 }
37
38 void uploadToThingSpeak(float temperature,
39                         float humidity) {
40   WiFiClientSecure client;
41
42   // Demonstration setting only. For production,
43   // use certificate validation instead.
44   client.setInsecure();
45
```

```
  HTTPClient http;
  Serial.println("Uploading data to ThingSpeak...");

  if (http.begin(client, SERVER_URL)) {
    http.addHeader("Content-Type",
                   "application/x-www-form-urlencoded");

    String postData = "api_key=" + String(WRITE_API_KEY);
    postData += "&field1=" + String(temperature, 2);
    postData += "&field2=" + String(humidity, 2);

    int responseCode = http.POST(postData);

    if (responseCode > 0) {
      String response = http.getString();

      Serial.print("HTTP Response Code: ");
      Serial.println(responseCode);

      Serial.print("ThingSpeak Entry ID: ");
      Serial.println(response);

      if (response.toInt() > 0) {
        Serial.println("SUCCESS: Data uploaded!");
      } else {
        Serial.println("Upload not accepted.");
      }
    } else {
      Serial.print("Upload Error: ");
      Serial.println(responseCode);
    }

    http.end();
  } else {
    Serial.println("Unable to connect to ThingSpeak!");
  }
}

void setup() {
  Serial.begin(115200);

```

```
  Serial.println();
  Serial.println("================================");
  Serial.println("    IoT WEATHER STATION");
  Serial.println("================================");

  dht.begin();
  Serial.println("DHT22 Sensor Initialized!");

  connectWiFi();
  Serial.println("System Ready!");
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
  }

  if (millis() - lastUploadTime >= UPLOAD_INTERVAL ||
      lastUploadTime == 0) {

    lastUploadTime = millis();

    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    if (isnan(temperature) || isnan(humidity)) {
      Serial.println("ERROR: Sensor reading failed!");
      return;
    }

    Serial.println();
    Serial.println("--------------------------------");
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" C");

    Serial.print("Humidity: ");
    Serial.print(humidity);
    Serial.println(" %");
    Serial.println("--------------------------------");

```

```
128       uploadToThingSpeak(temperature, humidity);
129     }
130
131     delay(100);
132   }
```

5.4 Library Dependencies

The Wokwi project uses the Adafruit DHT sensor library. The Adafruit Unified Sensor dependency should also be available in the project library configuration if required by the installed DHT library version.

CHAPTER 6

ThingSpeak Cloud Integration and Visualization

6.1 Channel Configuration

A ThingSpeak channel was created for the weather station. Two channel fields were enabled:

- **Field 1 – Temperature:** stores temperature measurements in degrees Celsius.
- **Field 2 – Humidity:** stores relative humidity measurements in percent.

6.2 Data Upload

The ESP32 firmware sends the two measurements to the ThingSpeak update endpoint. The Write API Key is included in the request so that ThingSpeak can identify the channel to be updated. A positive entry identifier in the response indicates that a new channel entry was created.

6.3 Data Visualization

ThingSpeak displays the stored measurements as separate time-series charts. The temperature graph uses Field 1 and the humidity graph uses Field 2. The dashboard allows the user to inspect how the virtual sensor values change over time.

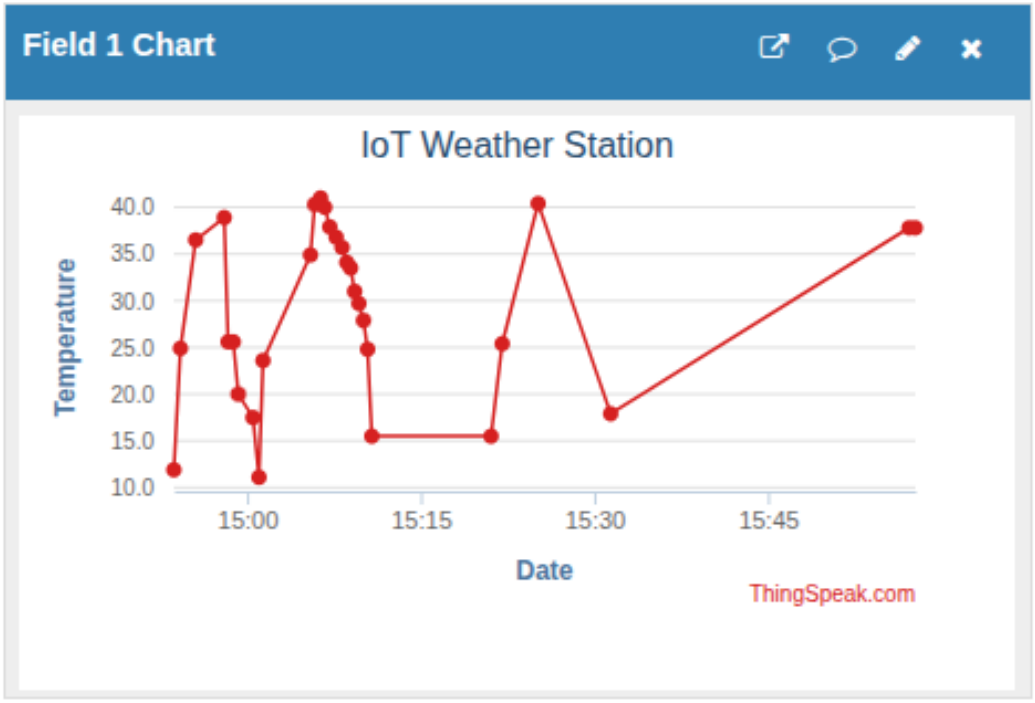


Figure 6.1: ThingSpeak temperature readings (Field 1).

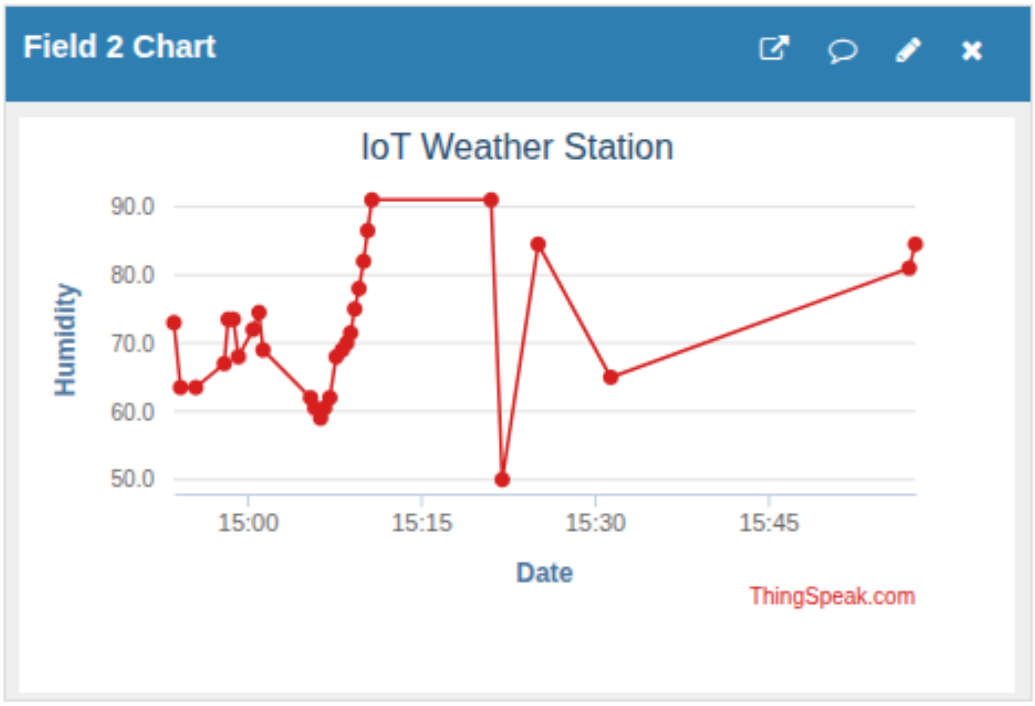


Figure 6.2: ThingSpeak relative humidity readings (Field 2).

6.4 Upload Interval

The firmware is configured to upload readings every 20 seconds. This interval was selected to remain above the commonly applied 15-second minimum update interval for a free ThingSpeak channel. The actual account and channel limits should always be checked against the current ThingSpeak plan.

CHAPTER 7

Email Alert Mechanism

7.1 Alert Conditions

The email alert mechanism is implemented using ThingSpeak Alerts API and MATLAB Analysis. Two alert conditions were configured:

Table 7.1: Configured alert thresholds

Parameter	Condition	Alert message
Temperature	Above 35 °C	High Temperature Alert
Relative humidity	Above 70%	High Humidity Alert

7.2 MATLAB Analysis

MATLAB Analysis is used to construct and send an alert request to the ThingSpeak Alerts API. The request includes a subject and message body. The Alerts API key is passed in the request header. The key is private and must not be included in public copies of the report.

7.3 Temperature Alert Code

The following is the alert-request pattern used for the temperature notification. The Alerts API key must be supplied privately when configuring the MATLAB Analysis.

Listing 7.1: Temperature alert request

```
alertApiKey = 'YOUR_ALERTS_API_KEY';
alertUrl = "https://api.thingspeak.com/alerts/send";

options = weboptions("HeaderFields", ...
    ["ThingSpeak-Alerts-API-Key", alertApiKey]);

mailSubject = ...
```

```
    "IoT Weather Station: High Temperature Alert";

mailBody = ...
    "WARNING: Temperature has exceeded 35 degrees Celsius.
       Please check the weather conditions.";

response = webwrite(alertUrl, ...
    "body", mailBody, ...
    "subject", mailSubject, options);

disp("Temperature alert email request accepted.");
```

7.4 Humidity Alert Code

Listing 7.2: Humidity alert request

```
alertApiKey = 'YOUR_ALERTS_API_KEY';
alertUrl = "https://api.thingspeak.com/alerts/send";

options = weboptions("HeaderFields", ...
    ["ThingSpeak-Alerts-API-Key", alertApiKey]);

mailSubject = ...
    "IoT Weather Station: High Humidity Alert";

mailBody = ...
    "WARNING: Relative humidity has exceeded 70 percent.
       Please check the weather conditions.";

response = webwrite(alertUrl, ...
    "body", mailBody, ...
    "subject", mailSubject, options);

disp("Humidity alert email request accepted.");
```

7.5 Alert API Rate Limit

During testing, HTTP status code 429 (Too Many Requests) was encountered when additional alert requests were made. This indicates that the alert service rate limit had

been reached. The project subsequently demonstrated successful receipt of both the high-temperature and high-humidity email notifications. The alert API quota is shared by requests made with the account's alert service; repeated manual testing should therefore be avoided.

7.6 Email Verification

Both alert email messages were received during the project test:

- High Temperature Alert: notification stating that temperature exceeded 35 °C.
- High Humidity Alert: notification stating that relative humidity exceeded 70%.

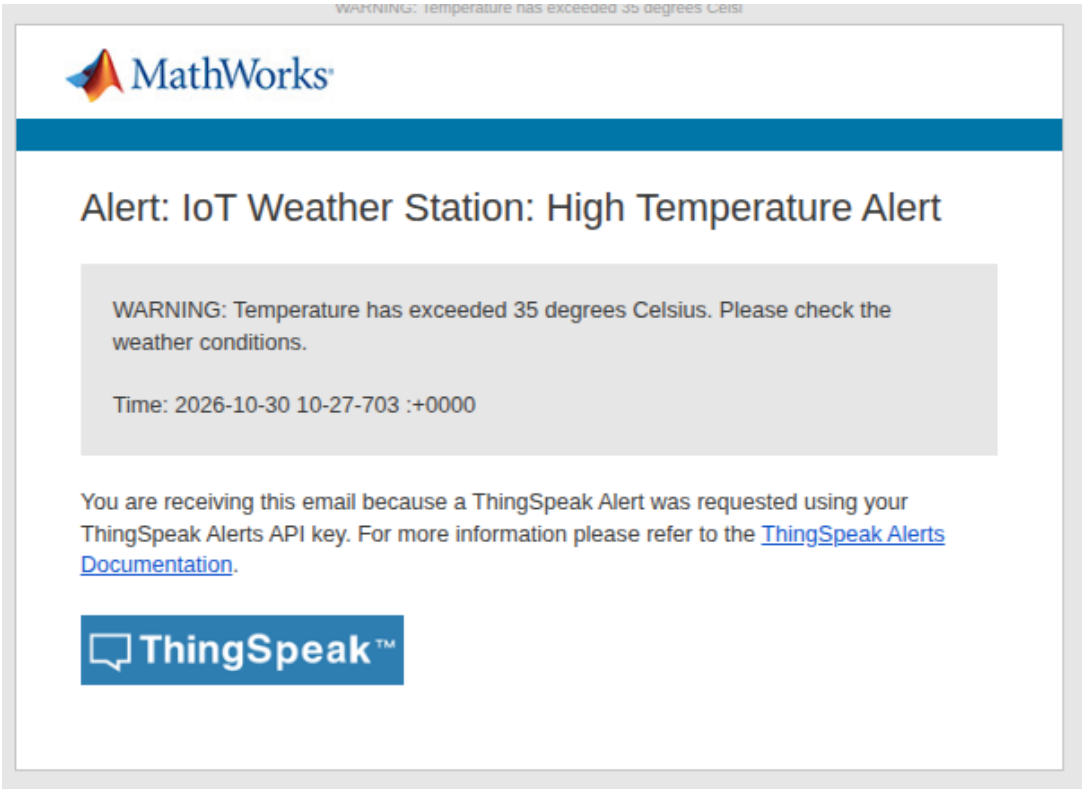


Figure 7.1: Received high-temperature email notification.

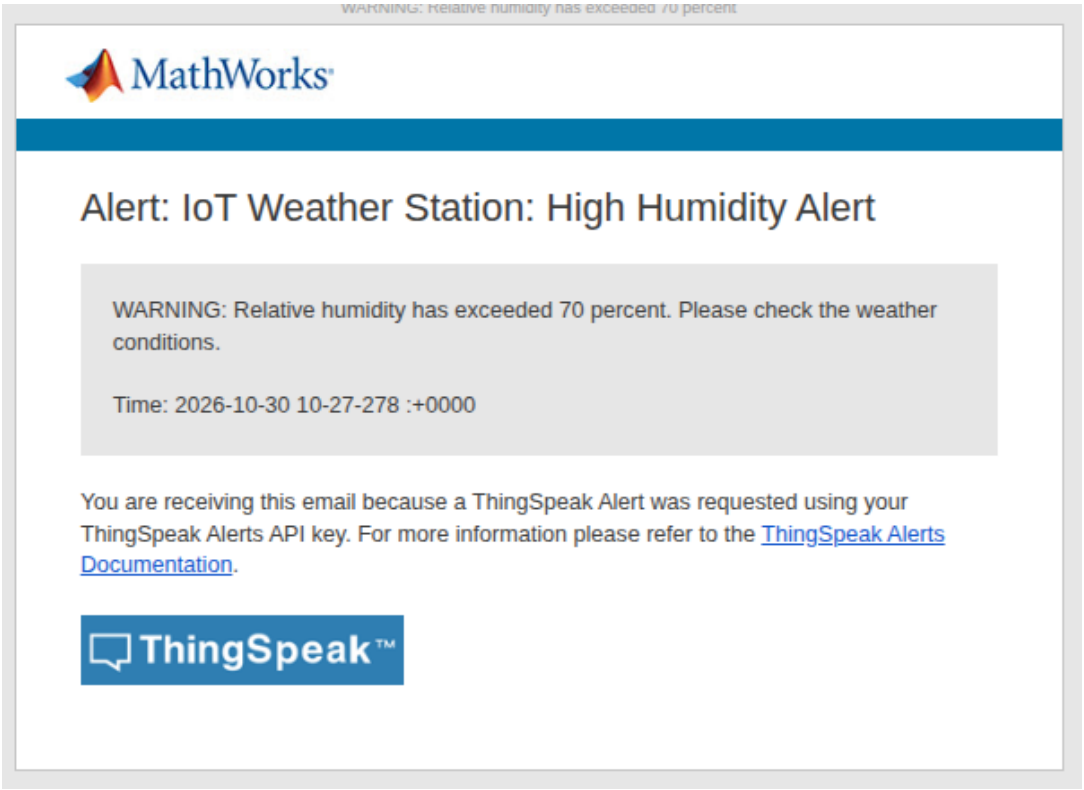


Figure 7.2: Received high-humidity email notification.

CHAPTER 8

Testing, Results and Discussion

8.1 Testing Methodology

The system was tested in Wokwi by running the ESP32 firmware with the virtual DHT22 sensor. The sensor's temperature and humidity controls were adjusted to produce different readings. The Serial Monitor was used to inspect sensor values, Wi-Fi connection status and ThingSpeak upload responses. The ThingSpeak channel was then checked to verify that the measurements appeared in the corresponding fields.

The alert mechanism was tested by causing the configured temperature and humidity thresholds to be exceeded and verifying receipt of the email notifications.

8.2 Functional Test Cases

Table 8.1: Functional test cases

No.	Test	Expected result	Observed result
1	DHT22 initialization	Sensor initializes without error	Successful in simulation
2	Temperature reading	Temperature appears in Serial Monitor	Successful
3	Humidity reading	Humidity appears in Serial Monitor	Successful
4	Wi-Fi connection	ESP32 connects to Wokwi-GUEST	Successful
5	ThingSpeak upload	New channel entry is created	Successful
6	Dashboard visualization	Field 1 and Field 2 graphs update	Successful

No.	Test	Expected result	Observed result
7	Temperature alert	High-temperature email is received	Successful
8	Humidity alert	High-humidity email is received	Successful

8.3 Results

The Wokwi simulation successfully demonstrated the acquisition of temperature and humidity values from the virtual DHT22 sensor. The ESP32 connected to the Wokwi virtual Wi-Fi network and uploaded the measurements to ThingSpeak. The ThingSpeak channel displayed separate graphs for temperature and humidity.

The email notification tests also succeeded. A high-temperature email and a high-humidity email were received after the corresponding alert requests were made. During repeated testing, the Alerts API returned HTTP 429 when its request limit was reached. This behaviour was treated as a service rate-limit response rather than a sensor or wiring fault.

8.4 Discussion

The project demonstrates that a simulated embedded device can communicate with an online IoT cloud platform and that the stored data can be visualized remotely. It also demonstrates an email notification workflow using MATLAB Analysis and the ThingSpeak Alerts API.

The results validate the functional flow of the simulation. However, they do not establish the accuracy of a physical DHT22 sensor, the reliability of a real Wi-Fi installation or the environmental durability of a deployed device.

8.5 Screenshots and Evidence

The project evidence images are included in the relevant sections of this report: the block diagram and circuit diagram in Chapters 2 and 4, the temperature and humidity charts in Chapter 6, and the two received alert emails in Chapter 7. The Wokwi Serial Monitor screenshot is attached here.

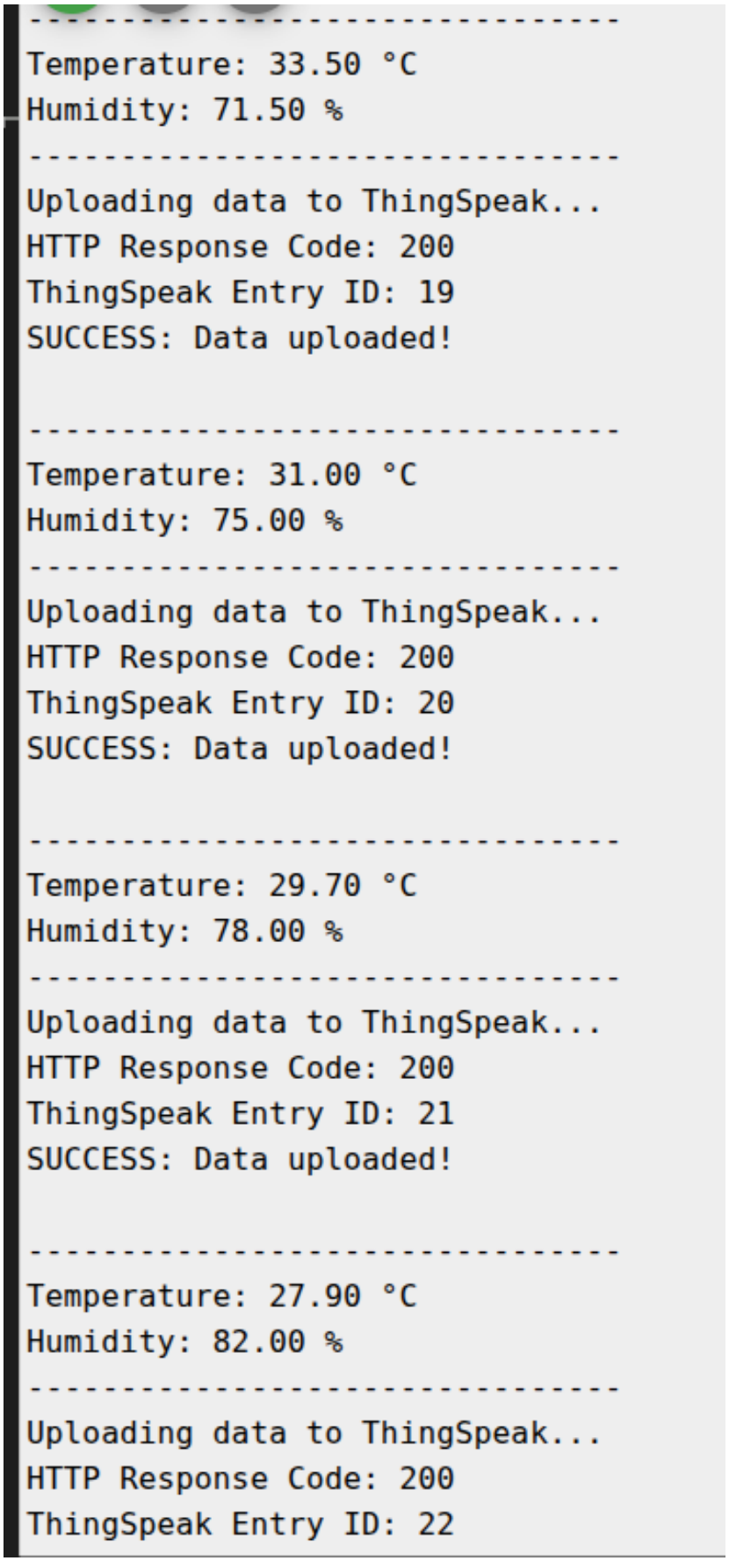


Figure 8.1: ESP32 Serial Monitor showing temperature, humidity, Wi-Fi connection and successful ThingSpeak data upload.

CHAPTER 9

Limitations and Future Scope

9.1 Limitations

- The project was implemented using Wokwi simulation rather than physical ESP32 and DHT22 hardware.
- Sensor accuracy and calibration were not tested against reference instruments.
- Network performance was demonstrated through the simulator's virtual Wi-Fi environment.
- The ThingSpeak Alerts API applies request limits, as observed through HTTP 429 during repeated tests.
- The firmware example uses `client.setInsecure()` for the HTTPS demonstration. This disables certificate verification and is not recommended for production deployment.

9.2 Future Enhancements

The project can be extended in several ways:

1. Implement the circuit using a physical ESP32 development board and DHT22 sensor.
2. Compare sensor readings with calibrated reference instruments.
3. Add sensors for atmospheric pressure, light intensity and rainfall.
4. Develop a mobile-friendly dashboard for remote monitoring.
5. Improve alert logic to prevent repeated notifications while a threshold remains exceeded.
6. Use certificate validation and secure credential storage for deployment.
7. Add local display and data logging for operation during network interruptions.

CHAPTER 10

Conclusion

The IoT-Based Weather Station Using ESP32 and Cloud Dashboard was successfully developed and demonstrated using the Wokwi simulation platform. The virtual DHT22 sensor was interfaced with the ESP32 through GPIO 15, and the firmware acquired temperature and relative humidity readings.

The ESP32 connected to the Wokwi-GUEST Wi-Fi network and transmitted the measurements to ThingSpeak. The cloud platform stored the data in separate temperature and humidity fields and displayed the measurements as graphs. The email notification mechanism was implemented using MATLAB Analysis and the ThingSpeak Alerts API. Both high-temperature and high-humidity alert emails were successfully received during testing.

The project therefore demonstrates the integration of sensor interfacing, embedded programming, Wi-Fi communication, cloud-based data visualization and email notification in an IoT application. Since the implementation was simulation-based, physical hardware testing remains a future activity.

References

[1] Wokwi, "Online Electronics Simulator." Available: https://wokwi.com/

[2] MathWorks, "ThingSpeak IoT Analytics Platform." Available: https://thingspeak.com/

[3] MathWorks, "ThingSpeak Alerts API Documentation." Available: https://www.mathworks.com/help/thingspeak/alerts-api.html

[4] MathWorks, "ThingSpeak MATLAB Analysis and React." Available: https://www.mathworks.com/help/thingspeak/

[5] Adafruit, "DHT Sensor Library." Available: https://github.com/adafruit/DHT-sensor-library

[6] Espressif Systems, "ESP32 Documentation." Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/